



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий

Кафедра математического обеспечения и стандартизации
информационных технологий

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 2

по дисциплине «Тестирование и верификация
программного обеспечения»

**Тема: «Модульное и мутационное тестирование
программного продукта»**

Команда «Дерби» · Дата выполнения: 09.10.2026

Состав: Дубейко А. С., Домбровский Я. А.,

Егшатын Г. А., Яхяев Ф. Ф. О., Лисенков А. М.

Выполнил:

Студент группы
ИКБО-61-24

Яхяев Фирамил Фируддин Оглы

Проверил:

Доцент кафедры МОСИТ,
кандидат технических наук, доцент
Чернов Е. А.

Москва 2026

Содержание

1 Цель и организация исследования.....	3
2 Приложение и объект проверки.....	4
3 Функции и проверяемые контракты.....	5
4 Модульные тесты и среда.....	6
5 Преднамеренный дефект и исправление.....	7
6 Покрытие строк и ветвей.....	8
7 Мутационный эксперимент.....	9
8 Анализ и выводы.....	12
Список использованных источников.....	13
Приложение А. Исходный код модели.....	14
Приложение Б. Тесты и журналы.....	17
Приложение В. Воспроизведение.....	19

Отдельная серия Y: временные контракты и жизненный цикл предметов.
Материалы: experiments_practice2_yakhyaev.zip.

1 Цель и организация исследования

Цель — оценить способность модульных тестов обнаруживать ошибки времени, траектории и очистки объектов в модели игры «Дерби у стадиона»; проверить исправление учебного дефекта и усилить тесты по результатам мутационного анализа.

Задачи: описать функции модели; подготовить независимый базовый набор; внести ошибку накопления возраста снаряда; зарегистрировать её проявление; восстановить корректное поведение; измерить покрытие; автоматически проверить новую выборку мутаций; объяснить выжившие варианты и добавить недостающие проверки.

Таблица 1 — Команда «Дерби», группа ИКБО-61-24

Участник	Область исследования
Дубейко Аким Сергеевич	Исходное исследование модели общего проекта.
Домбровский Ярослав Андреевич	Исходное исследование интерфейса общего проекта.
Егшатын Гайк Артакович	Отдельные сценарии меню и пользовательских событий.
Яхяев Фирамил Фируддин Оглы	Настоящий отчёт: кадры времени, полёт, анимационные таймеры и частицы.
Лисенков Андрей Михайлович	Отдельная серия проверок того же приложения.

Объектом служит готовый модуль общего проекта. Здесь описано индивидуальное исследование; создание всего модуля или личное выполнение проверок другими участниками не приписывается автору отчёта.

Материалы можно передать участнику команды для взаимной проверки: сохранены контракты, исходники, тесты, карточка дефекта и журналы. Эксперимент подтверждает автоматические запуски, а не факт личной проверки напарником.

2 Приложение и объект проверки

Игра позволяет выбрать обе стороны из десяти испанских клубов. При запуске фанаты «Барселоны» бросают предметы в персонажа «Реал Мадрид». Срасе запускает бросок, 1–3 выбирают предмет, Т открывает меню команд, R очищает раунд.

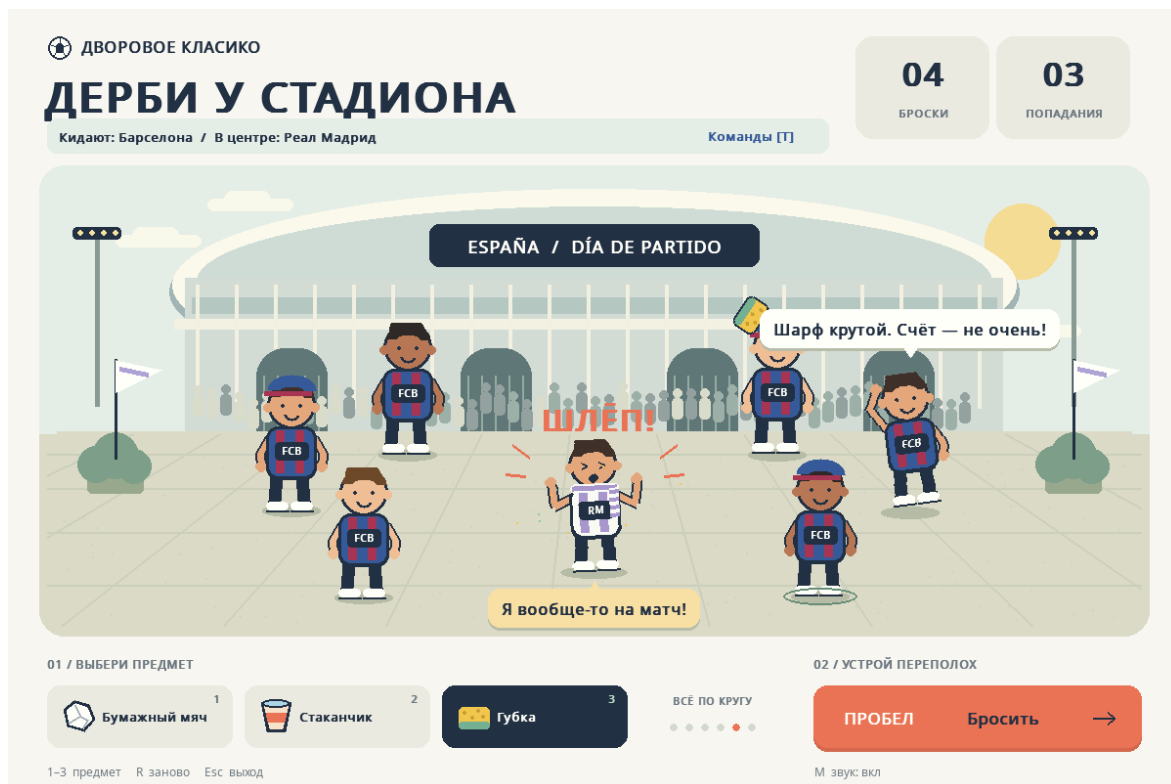


Рисунок 1 — Реальный безэкранный запуск обновлённой игры

Модуль model.py не использует окно и звук. GameState хранит текущий раунд, выбранные команды, летящие предметы и частицы. Метод update(dt) продвигает время на dt секунд. Благодаря этому логику можно проверять без Pygame, а пользовательский интерфейс — отдельным контрольным запуском.

Основной риск новой серии: проверка единственного большого шага не обнаруживает ошибку, которая проявляется только при нескольких последовательных кадрах. Поэтому полёт проверяется по четырём шагам по 0,2 с; успех означает ровно одно попадание после суммарных 0,8 с.

3 Функции и проверяемые контракты

Таблица 2 — Семь функций и свойств model.py

Функция / свойство	Контракт
Projectile.progress	Доля пройденного времени age/duration ограничена диапазоном [0; 1].
Projectile.position	Начало и конец совпадают с заданными координатами; середина траектории поднята на 145 логических единиц над линейной интерполяцией.
GameState.reset	Очищает счётчики, объекты и реплики; сохраняет выбранные команды.
GameState.choose_club	При выборе текущего соперника меняет стороны местами; корректный новый выбор очищает раунд; неверный или прежний выбор ничего не меняет.
GameState.select	Допустимы индексы предметов 0–2; иной индекс сохраняет предыдущий выбор.
GameState.throw	Создаёт один предмет при доступном броске; задержка 0,25 с; говорящий болельщик выбирается циклически.
GameState.update	Принимает конечное $dt \geq 0$; накапливает время полёта; регистрирует каждое попадание один раз; обновляет и удаляет частицы.

Траектория: $t = \min(1, \max(0, \text{age} / \text{duration}))$; $x = x_0 + (x_1 - x_0)t$; $y = y_0 + (y_1 - y_0)t - 580t(1 - t)$. Для (120; 430) $\rightarrow$ (620; 530) при $t = 0,5$ ожидается (370; 335).

Контракты анимации текущей версии: поза броска — 0,5 с; реплика бросающего — 2,4 с; реакция при попадании — 0,65 с; ответная реплика — 1,8 с; 15 частиц на попадание. Их соблюдение проверяется отдельными утверждениями.

4 Модульные тесты и среда

Наборы подготовлены специально для этого исследования. Для каждого теста создаётся новое состояние с `seed=41`. Предопределённый `seed` делает случайные координаты воспроизводимыми; ожидаемые значения времени, числа объектов и границ задаются контрактами, а не копированием случайного результата.

Таблица 3 — Фактические результаты исправного кода

Набор	Тестов	Провалов	Ошибок	Пропусков
Базовый	20	0	0	0
Улучшенный	40	0	0	0

Таблица 4 — Состав сценариев

Набор	Проверки
Базовый	Сумма неодинаковых кадров; полёт за четыре кадра; границы прогресса; начало/конец траектории; пауза броска; три предмета; <code>reset</code> ; неверное <code>dt</code> ; очистка частиц; цикл болельщиков.
Добавленный	Порядок незавершённых снарядов; точные независимые таймеры; 1–3 одновременных попадания; направление скоростей частиц; высота дуги; неверные индексы; обмен клубов; нулевой шаг и точное истечение жизни.

Таблица 5 — Среда выполнения

Параметр	Значение
Дата	09.10.2026
Python	3.12.14
Библиотеки	<code>pytest</code> 9.1.1; <code>coverage</code> 7.16.2; <code>pygame-ce</code> 2.5.8
Изоляция	Свежий временный каталог на каждый запуск; отдельный процесс <code>pytest</code> ; фиксированные версии библиотек.
Данные	Числа взяты из <code>JUnit</code> и <code>summary.json</code> ; параметризованные случаи считаются отдельными тестами.

5 Преднамеренный дефект и исправление

Таблица 6 — Карточка учебного дефекта

Поле	Содержание
Идентификатор	BUG-MODEL-04
Описание	Возраст предмета равен dt текущего кадра вместо накопленного времени; короткие кадры не завершают полёт.
Расположение	model.py, GameState.update, цикл self.projectiles
Категория / статус	Major; Open → Fixed → Closed
Предусловия	Новое GameState(seed=41); выполнен успешный throw().
Шаги	Четырежды вызвать update(0.2). Проверить число попаданий и остаток летящих предметов.
Ожидание	Вызовы возвращают 0, 0, 0, 1; hits=1; projectiles=[]; суммарное время 0,8 с.
Факт в дефектной копии	Все вызовы возвращают 0; hits=0; предмет остаётся в полёте с age=0,2 с.
Причина и исправление	Присваивание age = dt заменено на накопление age += dt.

Ошибка внесена намеренно в изолированную копию. Рабочая версия игры сохранена. Контрольный запуск с одним update(0.8) не выявляет эту ошибку, поэтому в базовом наборе используется последовательность кадров.

До исправления: 1 провал из 20 тестов; после исправления — 20 успешных тестов, ошибок 0. Заккрытие дефекта подтверждено повторным запуском исходного базового набора.

Дефект: projectile.age = dt
Исправление: projectile.age += dt

6 Покрывание строк и ветвей

Строки и ветви измеряются отдельно инструментом coverage.py с параметром --branch. Числитель — выполненные элементы, знаменатель — доступные исполняемые строки или исходы ветвления. Комбинированный процент coverage.py не подменяет ни один из этих показателей.

Таблица 7 — Покрывание всего model.py

Набор	Строки	Ветви
Базовый	118/121 (97,52%)	18/22 (81,82%)
Улучшенный	121/121 (100,00%)	22/22 (100,00%)
Улучшенный + SDL	121/121 (100,00%)	22/22 (100,00%)

Базовый набор не выполняет строки 102, 104 и 107: отказ при неверном или неизменившемся клубе и обмен сторон. Также пропущен отрицательный исход проверки индекса предмета. Улучшенный набор добавляет эти случаи: в model.py после него не остаётся непокрытых строк и ветвей.

При базовом покрытии строк 97,52% выжили восемь из двенадцати мутантов: множество строк выполнялось, но тесты не сравнивали точный порядок объектов, число частиц и длительность анимации. Это показывает различие выполнения кода и точности проверок.

Дополнительный SDL-запуск проверяет соединение модели с реальной игрой и сохраняет скриншоты. Он не увеличил уже полное покрытие model.py. Показатели графического game.py не заявляются показателями модульных тестов модели.

Границы исследования: фиксированная выборка сценариев, положительная штатная длительность полёта и поддерживаемые индексы объектов. Реальное аудиоустройство и оконное поведение macOS в безэкранный режим не проверяются.

7 Мутационный эксперимент

Скрипт `run_experiments.py` автоматически создаёт ограниченную выборку из двенадцати новых изменений Y01–Y12. Каждый мутант содержит одну замену в `model.py`. Проверяется единственность исходного фрагмента и корректность синтаксиса; затем вариант запускается отдельным процессом в новом временном каталоге.

Сначала исправная модель проходит базовый набор. Если мутант также проходит все тесты, он считается выжившим. Если `pytest` сообщает провал утверждения, мутант обнаружен. Ошибки сбора, ошибки запуска, пустой набор и тайм-аут не засчитываются как обнаружение.

После анализа выживших вариантов добавлены точные проверки времени и состояния. Та же выборка повторно запускается на улучшенном наборе. Все `baseline`-тесты сохраняются; рабочий код приложения не изменяется для подгонки под мутантов.

$\text{Mutation score} = \text{обнаруженные} / \text{валидные мутанты} \times 100\%$. До усиления: $4/12 = 33,33\%$. После: $12/12 = 100,00\%$. Все 24 запуска мутаций завершились без ошибок и тайм-аутов.

Учебный дефект BUG-MODEL-04 учитывается отдельно и не включён в двенадцать мутантов. Операторы: AOR — изменение арифметики; ROR — изменение сравнения; CRP — замена значения; CALL — замена операции с коллекцией в данной учебной классификации.

Это собственный прозрачный генератор, а не MutPy или универсальный перебор всех операторов программы. Он ограничен явными правилами; поэтому 100% означает обнаружение данной выборки, а не доказательство отсутствия дефектов во всём приложении.

7.1 Полная выборка и результаты

Таблица 8 — Мутации модели Y01–Y12

ID	Изменение	До	После
Y01	Часы раунда заменяют накопление последним кадром	Обнаружен	Обнаружен
Y02	Снаряд летит вдвое быстрее	Обнаружен	Обнаружен
Y03	Попадание не происходит на границе полёта	Обнаружен	Обнаружен
Y04	Одно попадание засчитывается дважды	Обнаружен	Обнаружен
Y05	Меняется порядок ещё летящих предметов	Выжил	Обнаружен
Y06	Реакция персонажа короче заданной	Выжил	Обнаружен
Y07	Ответная реплика исчезает раньше	Выжил	Обнаружен
Y08	Поза броска исчезает раньше	Выжил	Обнаружен
Y09	Реплика бросающего исчезает раньше	Выжил	Обнаружен
Y10	Создаётся на одну частицу меньше	Выжил	Обнаружен
Y11	Частица движется против горизонтальной скорости	Выжил	Обнаружен
Y12	Частица движется против вертикальной скорости	Выжил	Обнаружен

«До» — базовые 20 тестов; «После» — улучшенные 40 тестов. Для каждого варианта сохранены исходник, правило замены, результат, полный stdout и XML JUnit.

Y01–Y04 выявлены сразу: базовые тесты наблюдают накопление времени, границу завершения и счёт попаданий. Остальные изменения не нарушают эти крупные исходы, поэтому для их обнаружения потребовались дополнительные контракты.

7.2 Анализ выживших вариантов

Таблица 9 — Пробелы и усиление утверждений

ID	Причина	Новая проверка
Y05	Не сравнивался порядок одновременно летящих предметов.	test_unfinished_projectiles_keep_creation_order: сравнить список после короткого кадра.
Y06	Фиксировалось попадание, но не продолжительность реакции.	test_reaction_and_reply_use_their_own_lifetimes: 0,65 с и уменьшение после кадра.
Y07	Не проверялась длительность ответной реплики.	Тот же тест: 1,8 с и независимое уменьшение таймера.
Y08	Не измерялась поза бросающего болельщика.	test_throw_pose_and_bubble_have_independent_timers: 0,5 с и остаток 0,25 с.
Y09	Не измерялась реплика бросающего.	Тот же тест: 2,4 с и остаток 2,15 с.
Y10	Проверялось только наличие частиц.	test_simultaneous_impacts_create_fifteen_particles_each: ровно 15 на одно попадание.
Y11	Не проверялось перемещение частиц по горизонтали.	test_particle_moves_in_velocity_direction_before_gravity: обе скорости и точная координата x.
Y12	Не проверялось перемещение частиц по вертикали.	Тот же тест: обе скорости, точная координата y и ускорение.

Все восемь вариантов после усиления дают реальные провалы утверждений. У них наблюдаемое отличие от контрактов исправной версии, поэтому исключать их как эквивалентные нет оснований.

8 Анализ и выводы

Для нового исследования модели описаны семь функций и свойств. Созданы 20 базовых и 40 улучшенных тестовых случаев. Исправная версия прошла оба набора без ошибок и пропусков.

Учебная ошибка накопления возраста обнаружена при четырёх последовательных кадрах. После восстановления операции += повторная проверка прошла успешно. Данный сценарий полезен тем, что одиночный большой шаг времени мог скрыть дефект.

На новой выборке из двенадцати мутантов эффективность выросла с 33,33% до 100,00%. Усилены проверки порядка объектов, независимых таймеров и физических параметров частиц. Все улучшения проверены реальными повторными запусками.

Покрытие модели выросло со 118/121 строк и 18/22 ветвей до 121/121 и 22/22. При этом исходное высокое покрытие строк не предотвращало выживание восьми мутантов. Следовательно, существенны утверждения о результате, а не только выполнение оператора.

Отчёт и архив позволяют другому участнику воспроизвести эксперимент и проверить исправление. Результаты не выдаются за независимую ручную работу другого студента. Готовый модуль является частью общего проекта, а данная серия имеет собственные сценарии и журналы.

Дальнейшая работа: длительные последовательности бросков и смены клубов, генерация разных наборов длительностей кадров, ограничения нагрузки на коллекции и ручная проверка интерфейса. Полное покрытие текущих строк и обнаружение конечной выборки мутаций не гарантируют отсутствие всех ошибок.

Список использованных источников

1. Практическая работа № 2. Модульное и мутационное тестирование программного продукта. Предоставленные методические указания, разделы 2.3–2.4.

2. ГОСТ 19.401-78. Единая система программной документации. Текст программы. Требования к содержанию и оформлению.

3. ГОСТ 34.602-2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.

4. pytest documentation. <https://docs.pytest.org/>

5. coverage.py documentation. <https://coverage.readthedocs.io/>

6. pygame-ce documentation. <https://pyga.me/docs/>

7. Python documentation. math, random, dataclasses.
<https://docs.python.org/3/library/>

8. Исходный проект «Дерби у стадиона». <https://github.com/akimdubeyko-dotcom/calculator>

Дата обращения к электронным ресурсам: 09.10.2026.

Приложение А. Исходный код модели

Ниже приведён полный исправный model.py. Копия и SHA-256 сохранены в архиве, чтобы связать эксперимент с точной версией программы.

```
"""Deterministic game simulation, independent of the display and sound system."""

from dataclasses import dataclass
import math
import random

FANS = ((290, 512), (409, 455), (787, 455), (923, 514), (831, 600), (364, 601))
TARGET = (600, 501)
ITEMS = ("Бумажный мяч", "Стаканчик", "Губка")
@dataclass(frozen=True)
class Club:
    name: str
    badge: str
    primary: tuple[int, int, int]
    secondary: tuple[int, int, int]

CLUBS = (
    Club("Барселона", "FCB", (54, 90, 153), (167, 52, 83)),
    Club("Реал Мадрид", "RM", (250, 250, 244), (168, 151, 205)),
    Club("Атлетико Мадрид", "ATM", (225, 58, 65), (250, 250, 244)),
    Club("Севилья", "SEV", (250, 250, 244), (206, 45, 58)),
    Club("Реал Бетис", "BET", (36, 151, 91), (250, 250, 244)),
    Club("Валенсия", "VAL", (250, 250, 244), (235, 130, 47)),
    Club("Вильярреал", "VIL", (246, 216, 62), (62, 92, 158)),
    Club("Атлетик Бильбао", "ATH", (212, 52, 59), (250, 250, 244)),
    Club("Реал Сосьедад", "RSO", (54, 111, 186), (250, 250, 244)),
    Club("Жирона", "GIR", (218, 53, 65), (250, 250, 244)),
)
TAUNTS = (
    "Эй, лови стакан!",
    "{target}, где твоя защита?",
    "Эй, растяпа, лови!",
    "{throwers} передаёт привет!",
    "Шарф крутой. Счёт — не очень!",
    "До встречи на поле!",
)
REPLIES = ("Наши ещё покажут!", "Судья! Это фол!", "Я вообще-то на матч!", "Ещё увидимся на поле!")

@dataclass
class Projectile:
    start: tuple[float, float]
    end: tuple[float, float]
    kind: int
    duration: float = 0.8
    age: float = 0.0

    @property
    def progress(self):
        return min(1.0, max(0.0, self.age / self.duration))

    @property
    def position(self):
        t = self.progress
        return (
            self.start[0] + (self.end[0] - self.start[0]) * t,
            self.start[1] + (self.end[1] - self.start[1]) * t - 145 * 4 * t * (1 - t),
        )

@dataclass
class Particle:
    x: float
    y: float
    vx: float
    vy: float
    life: float
    color: tuple[int, int, int]
```

Приложение А — состояние, выбор и бросок

```
class GameState:
    cooldown_seconds = 0.25

    def __init__(self, seed=None):
        self.rng = random.Random(seed)
        self.throwers = 0
        self.target = 1
        self.reset()

    def reset(self):
        self.time = 0.0
        self.selected = 0
        selfthrows = 0
        self.hits = 0
        self.cooldown = 0.0
        self.reaction = 0.0
        self.projectiles = []
        self.particles = []
        self.poses = [0.0] * len(FANS)
        self.speaker = 0
        self.bubble = ""
        self.bubble_time = 0.0
        self.reply = ""
        self.reply_time = 0.0
        self.last_kind = 0

    def choose_club(self, role, index):
        """Change a side, swapping opponents if necessary, and start a fresh round."""
        if role not in ("throwers", "target") or index not in range(len(CLUBS)):
            return False
        if getattr(self, role) == index:
            return False
        other = "target" if role == "throwers" else "throwers"
        if getattr(self, other) == index:
            setattr(self, other, getattr(self, role))
        setattr(self, role, index)
        self.reset()
        return True

    def select(self, kind):
        if kind in range(len(ITEMS)):
            self.selected = kind

    def throw(self):
        """Return whether a new throw was accepted (rapid presses have a cooldown)."""
        if self.cooldown > 1e-9:
            return False
        self.speaker = selfthrows % len(FANS)
        x, y = FANS[self.speaker]
        direction = 1 if x < TARGET[0] else -1
        self.projectiles.append(Projectile(
            (x + direction * 29, y - 88),
            (TARGET[0] + self.rng.uniform(-15, 15), TARGET[1] + self.rng.uniform(-10, 10)),
            self.selected,
        ))
        self.poses[self.speaker] = 0.5
        self.bubble = TAUNTS[0] if self.selected == 1 else self.rng.choice(TAUNTS[1:])
        self.bubble = self.bubble.format(target=CLUBS[self.target].name,
throwers=CLUBS[self.throwers].name)
        self.bubble_time = 2.4
        selfthrows += 1
        self.cooldown = self.cooldown_seconds
        return True
```

Приложение А — обновление симуляции

```
def update(self, dt):
    """Advance elapsed seconds and return the number of new impacts."""
    if not math.isfinite(dt) or dt < 0:
        raise ValueError("dt must be a finite, nonnegative number")
    self.time += dt
    for name in ("cooldown", "reaction", "bubble_time", "reply_time"):
        setattr(self, name, max(0.0, getattr(self, name) - dt))
    self.poses = [max(0.0, pose - dt) for pose in self.poses]
    for particle in self.particles:
        particle.life -= dt
        particle.x += particle.vx * dt
        particle.y += particle.vy * dt
        particle.vy += 460 * dt
    self.particles = [p for p in self.particles if p.life > 0]
    impacts = 0
    pending = []
    for projectile in self.projectiles:
        projectile.age += dt
        if projectile.progress < 1:
            pending.append(projectile)
            continue
        impacts += 1
        self.hits += 1
        self.reaction = 0.65
        self.last_kind = projectile.kind
        self.reply = self.rng.choice(REPLIES)
        self.reply_time = 1.8
        palette = (
            ((248, 245, 223), (211, 218, 227)),
            ((92, 184, 229), (164, 220, 243)),
            ((246, 196, 81), (131, 202, 183)),
        )[projectile.kind]
        for _ in range(15):
            angle = self.rng.uniform(-math.pi, 0)
            speed = self.rng.uniform(65, 240)
            self.particles.append(Particle(
                *projectile.end, math.cos(angle) * speed, math.sin(angle) * speed,
                self.rng.uniform(0.3, 0.7), self.rng.choice(palette),
            ))
    self.projectiles = pending
    return impacts
```

SHA-256 исправного model.py:

a6db5acd5baf2553b53b5d5035776abed44c77725c63a9f4eed6ca2d4d1c816c4

Зависимый game.py включён в src/ для контрольного запуска настоящего интерфейса. Полные исходники тестов находятся в tests/.

Приложение Б. Тесты и журналы

tests/baseline/test_lifecycle.py

```
def test_flight_finishes_after_four_short_frames():
    state = GameState(seed=41)
    assert state.throw()
    assert state.update(0.2) == 0
    assert state.update(0.2) == 0
    assert state.update(0.2) == 0
    assert state.hits == 0
    assert state.update(0.2) == 1
    assert state.hits == 1
    assert state.projectiles == []
    assert state.particles
    assert state.update(0) == 0
    assert state.hits == 1
```

tests/improved/test_temporal_contracts.py

```
def test_unfinished_projectiles_keep_creation_order():
    state = GameState(seed=41)
    first = Projectile((10, 200), (300, 400), 0)
    second = Projectile((30, 200), (300, 400), 1)
    third = Projectile((50, 200), (300, 400), 2)
    state.projectiles = [first, second, third]
    state.update(0.125)
    assert state.projectiles == [first, second, third]
    assert [p.age for p in state.projectiles] == [0.125] * 3
```

tests/improved/test_temporal_contracts.py

```
@pytest.mark.parametrize('vx,vy', [(80, -120), (-80, 120), (0, 0)])
def test_particle_moves_in_velocity_direction_before_gravity(vx, vy):
    state = GameState(seed=41)
    state.particles = [Particle(200, 300, vx, vy, 2, (1, 2, 3))]
    state.update(0.25)
    particle = state.particles[0]
    assert (particle.x, particle.y) == pytest.approx((200 + vx / 4, 300 + vy / 4))
    assert particle.vy == pytest.approx(vy + 115)
    assert particle.life == pytest.approx(1.75)
```

Показаны ключевые примеры. Остальные случаи, включая все параметризации и импорты, входят в полные файлы архива.

Приложение Б — фактические журналы

Базовая корректная версия

```
20 passed in 0.07s
/workspace/calculator/.venv/lib/python3.12/site-packages/coverage/inorout.py:558:
CoverageWarning: Module game was never imported. (module-not-imported); see
https://coverage.readthedocs.io/en/7.16.2/messages.html#warning-module-not-imported
self.warn(f"Module {pkg} was never imported.", slug="module-not-imported")
```

Улучшенная корректная версия

```
40 passed in 0.12s
/workspace/calculator/.venv/lib/python3.12/site-packages/coverage/inorout.py:558:
CoverageWarning: Module game was never imported. (module-not-imported); see
https://coverage.readthedocs.io/en/7.16.2/messages.html#warning-module-not-imported
self.warn(f"Module {pkg} was never imported.", slug="module-not-imported")
```

Дефект до исправления

```
.F..... [100%]
===== FAILURES =====
_____ test_flight_finishes_after_four_short_frames _____
tests/test_lifecycle.py:21: in test_flight_finishes_after_four_short_frames
    assert state.update(0.2) == 1
E   assert 0 == 1
E   + where 0 = update(0.2)
E   +     where update = <model.GameState object at 0x7f5d36683680>.update
===== short test summary info =====
FAILED tests/test_lifecycle.py::test_flight_finishes_after_four_short_frames
1 failed, 19 passed in 0.04s
```

Повторный запуск исправленной версии

```
..... [100%]
20 passed in 0.04s
```

Полные логи и JUnit находятся в каталогах results/. Приведённые строки получены запуском, а не оформлены как предполагаемый результат.

Приложение В. Воспроизведение

Распаковать experiments_practice2_yakhyaev.zip и открыть терминал в каталоге с run_experiments.py. На Mac с Python 3.15 выполнить:

```
python3.15 -m venv .venv  
.venv/bin/python -m pip install -r requirements.txt  
.venv/bin/python run_experiments.py
```

На другой системе использовать установленный python3. Фактические измерения этого отчёта получены на Python 3.12.14; версии зависимостей зафиксированы. Время выполнения и оформление шрифтов могут отличаться между системами.

Таблица В.1 — Материалы для воспроизведения

Путь	Содержание
src/	Исправные model.py и game.py.
tests/baseline/	Новые 20 базовых случаев.
tests/improved/	Все базовые и 20 дополнительных случаев.
run_experiments.py	Правила дефекта и двенадцати мутаций; запуск процессов.
requirements.txt	Фиксированные версии зависимостей.
results/summary.json	Версии, SHA-256, исходы всех запусков.
results/BUG-MODEL-04-*/	Проверки дефектной и восстановленной версий.
results/baseline-Y*/; improved-Y*/	Копии мутантов, stdout, JUnit, outcome.json.
results/*/htmlcov/	Отчёты покрытия строк и ветвей.
results/integration/	Контрольный запуск, скриншоты, интегральное покрытие.

Серия Y отделена от предыдущих исследований. Учебные мутанты предназначены только для эксперимента; устанавливать их вместо рабочей игры не требуется.